

Documentação do Trabalho Prático 3: A Equipe do Papai Noel

Felipe Almeida Pereira - 2024080051

Novembro 2025

1 Introdução

O problema computacional apresentado consiste em auxiliar o Papai Noel na formação de uma equipe de duendes para o Natal. O cenário envolve um conjunto de N duendes, onde existem relações de rivalidade entre alguns pares. O objetivo é selecionar o maior grupo possível de duendes tal que nenhum par dentro do grupo tenha uma relação de rivalidade (briga).

Além de maximizar o tamanho da equipe, o problema impõe um critério de desempate: caso existam múltiplas equipes com o tamanho máximo, deve-se escolher aquela que for lexicograficamente menor quando os índices dos duendes são listados em ordem crescente.

Em termos de Teoria dos Grafos, este é o clássico problema do Conjunto Independente Máximo (*Maximum Independent Set*). Dado que N pode chegar a 40, uma abordagem de força bruta pura $O(2^N)$ seria inviável.

2 Modelagem

O problema foi modelado como um grafo não direcionado $G = (V, E)$ onde:

- V representa os N duendes numerados de 0 a $N - 1$.
- E representa as relações de rivalidade. Uma aresta (u, v) existe se, e somente se, os duendes u e v brigam entre si.

Para armazenar essas relações, foi utilizada uma matriz de adjacência $adj[N][N]$, permitindo verificar se dois duendes entram em conflito em tempo constante $O(1)$. Como o limite máximo é $N = 40$, a matriz ocupa espaço pequeno ($40 \times 40 = 1600$ posições) e apresenta desempenho superior ao de listas de adjacência quando combinada com operações bitwise.

Dado que o problema busca o Conjunto Independente Máximo, que é NP-difícil, e considerando o limite de $N \leq 40$, uma abordagem tradicional de força bruta $O(2^N)$ seria inviável. Assim, o conjunto de vértices foi dividido em duas metades de tamanho $N/2$, permitindo a aplicação da técnica Meet-in-the-Middle, reduzindo a complexidade para aproximadamente:

$$O(N \cdot 2^{N/2})$$

3 Solução

A solução foi baseada em duas fases principais:

1. cálculo da melhor equipe possível dentro do Grupo 2 utilizando Programação Dinâmica, e
2. combinação dos melhores resultados do Grupo 1 com os compatíveis do Grupo 2.

Divisão dos duendes

- Grupo 1: índices de 0 a $(N/2) - 1$
- Grupo 2: índices de $N/2$ a $N - 1$

3.1 Etapa de Programação Dinâmica (Grupo 2)

A primeira etapa consiste em pré-calcular a melhor equipe possível para cada subconjunto de duendes do Grupo 2. Utilizamos uma tabela de Programação Dinâmica (`dp_grupo2`) indexada por uma máscara de bits (*bitmask*), onde cada bit representa a presença de um duende do Grupo 2 no subconjunto considerado.

Por que é Programação Dinâmica?

A solução se enquadra no paradigma de Programação Dinâmica pois apresenta subestrutura ótima e sobreposição de subproblemas. Para encontrar a melhor equipe dentro de uma máscara S , analisamos o duende de menor índice i em S . Temos duas escolhas ótimas locais:

1. Não incluir i : a solução é a mesma encontrada para o subconjunto $S \setminus \{i\}$ (já calculada).
2. Incluir i : a solução é o duende i somado à melhor equipe do subconjunto formado pelos duendes restantes em S que não brigam com i .

Ao armazenar os resultados na tabela `dp_grupo2`, evitamos recalcular as melhores equipes para as mesmas combinações de duendes repetidamente.

Pseudocódigo (Grupo 2)

para `mask` de 0 até $(1 \ll n2) - 1$:

i = menor índice ativo em `mask`

 se não existe i :

`dp_grupo2[mask]` = conjunto vazio

 senão:

`sol_excluindo` = `dp_grupo2[mask sem i]`

 se incluir i não causa conflito:

```

nova_mask = mask removendo i e todos que brigam com i
sol_incluindo = {i} dp_grupo2[nova_mask]
dp_grupo2[mask] = melhor(sol_incluindo, sol_excluindo)
caso contrário:
dp_grupo2[mask] = sol_excluindo

```

3.2 Etapa de Combinação (Grupo 1)

Na segunda etapa, o algoritmo itera por todas as máscaras possíveis do Grupo 1. Para cada máscara:

1. Verifica-se se é um conjunto independente válido (sem brigas internas).
2. Identifica-se quais duendes do Grupo 2 são compatíveis (não brigam com ninguém da equipe atual do Grupo 1).
3. Consulta-se a tabela de PD para obter instantaneamente a melhor equipe complementar formada pelos compatíveis.
4. Combina-se as duas partes e verifica-se se a nova equipe é a melhor encontrada.

Em caso de empate no tamanho da equipe, a comparação lexicográfica é utilizada para escolher o conjunto com menor ordem crescente, garantindo aderência total às regras do enunciado.

Pseudocódigo Final

```
melhor_resposta = vazio
```

```
para cada mask1 em 0..(1 << n1)-1:
```

```
  se mask1 é independente:
```

```
    compativel = máscara com todos os duendes do Grupo 2 compatíveis
```

```
    melhor_grupo2 = dp_grupo2[compativel]
```

```
    candidato = unir(mask1, melhor_grupo2)
```

```
  se candidato é maior que melhor_resposta:
```

```
    melhor_resposta = candidato
```

```
  senão se empatou em tamanho e é lexicograficamente menor:
```

```
    melhor_resposta = candidato
```

4 Análise de Complexidade

Seja N o número total de duendes. O algoritmo divide o problema em dois grupos de tamanho $N/2$.

Complexidade de Tempo

- A construção da tabela de PD para o Grupo 2 percorre $2^{N/2}$ estados.
- A varredura do Grupo 1 também percorre $2^{N/2}$ estados.

Logo:

$$O(N \cdot 2^{N/2})$$

Para $N = 40$, temos $40 \cdot 2^{20} \approx 4 \cdot 10^7$ operações, plenamente executáveis dentro de limites típicos de tempo.

Complexidade de Memória

$$O(N \cdot 2^{N/2})$$

5 Considerações Finais

A realização deste trabalho foi fundamental para compreender técnicas de otimização para problemas NP-difíceis com entradas limitadas. A maior dificuldade inicial foi a restrição de $N = 40$, que eliminava soluções de força bruta simples.

A implementação da técnica Meet-in-the-Middle foi desafiadora, especialmente na manipulação correta das máscaras de bits e na garantia da ordem lexicográfica durante a fusão dos vetores. Um ponto de aprendizado interessante foi a análise do Exemplo 01, onde a confiança na lógica do algoritmo permitiu identificar uma inconsistência no caso de teste manual do enunciado, reforçando a importância de seguir as especificações formais do problema acima dos exemplos visuais.

6 Referências

- CORMEN, T. H. et al. *Introduction to Algorithms*. 3rd ed. MIT Press, 2009.
- GEEKSFORGEES. *Maximum Independent Set using Meet-in-the-middle*. Disponível em: <https://www.geeksforgeeks.org/>. Acesso em: Nov. 2025.
- CPPREFERENCE. Documentação padrão C++. Disponível em: <https://en.cppreference.com/>. Acesso em: Nov. 2025.